

Automation Home Exercise #2: User Authentication System

Overview [↗](#)

This exercise is designed to test your understanding of full-stack development, focusing on user authentication and basic frontend-backend integration. You will create a simple web application with user registration, login functionality, and a basic dashboard.

Technical Requirements [↗](#)

Backend (Python) [↗](#)

- Use Python with Flask or FastAPI framework
- Implement user registration and login endpoints
- Implement password hashing for security
- Create JWT (JSON Web Token) authentication
- Include basic input validation
- Write unit tests for your authentication logic

Frontend [↗](#)

- Create a simple login page with HTML, CSS, and JavaScript
- Implement form validation
- Handle API calls to the backend
- Store JWT token in localStorage
- Create a simple dashboard page that's only accessible after login
- Make the UI responsive and user-friendly

Required Features [↗](#)

User Registration [↗](#)

- Email and password fields
- Password confirmation
- Basic password strength requirements
- Display appropriate error messages

User Login [↗](#)

- Email and password fields
- Remember me functionality (optional)
- Error handling for invalid credentials
- Redirect to dashboard after successful login

Dashboard [↗](#)

- Display user information
- Logout functionality
- Protected route (only accessible when logged in)

Technical Stack [↗](#)

- Backend: Python (Flask/FastAPI)
- Frontend: HTML, CSS, JavaScript
- Authentication: JWT

Project Structure [🔗](#)

```
1 project/
2 |─ backend/
3 |   |─ app.py
4 |   |─ models.py
5 |   |─ routes.py
6 |   |─ config.py
7 |   └─ tests/
8 |─ frontend/
9 |   |─ index.html
10 |   |─ dashboard.html
11 |   |─ styles.css
12 |   └─ script.js
13 └─ README.md
14
```

Evaluation Criteria [🔗](#)

1. Code Organization and Structure
 - Clean code principles
 - Proper file organization
 - Clear documentation
2. Functionality
 - All required features implemented
 - Error handling
 - Security best practices
3. User Experience
 - Intuitive interface
 - Responsive design
 - Clear error messages
4. Technical Implementation
 - Proper authentication flow
 - Secure password handling
 - API design and implementation

Bonus Points [🔗](#)

- Implement database storage for user data (SQLite or any other database)
- Implement password reset functionality
- Add email verification
- Include user profile editing
- Add remember me functionality
- Implement rate limiting for login attempts
- Submit the solution using Git repository with proper commit history

Submission Requirements [🔗](#)

1. README.md with:
 - Setup instructions

- API documentation
 - Testing instructions
2. Screenshots of the application
 3. Any additional notes or assumptions

Time Frame [↗](#)

- Expected completion time: 1 week
- Please document any challenges faced and how you overcame them

Getting Started [↗](#)

1. Set up your development environment
2. Create the project structure
3. Implement the backend first
4. Create the frontend
5. Test thoroughly
6. Document your work

Resources [↗](#)

- [Flask Documentation](#)
- [FastAPI Documentation](#)
- [JWT Documentation](#)
- [MDN Web Docs](#)

Good luck! We're looking forward to seeing your implementation.